

AgilA32

Developed by Alexander Co Abad with the help of Claude AI (Sonnet 5.5)

A from-scratch RV32I CPU with RP2040-compatible PIO state machines on one die: a firmware-programmable protocol emulator for IHP SG13G2 / Tiny Tapeout (6x4 tiles)

Document	Project documentation for review
Author	Alexander Co Abad, Department of Electronics, Computer and Electrical Engineering, De La Salle University, Manila
Repository	github.com/alexandercoabad/AgilA32_PIO (snapshot of 5 Oct 2026, features 1-20)
Purpose	Entry material for the Jane Street protocol-emulator ASIC competition (submission deadline 18 Jan 2027)
Target	IHP SG13 CMOS5L via Tiny Tapeout, 6x4 tiles, 24 MHz, open-source flow (Yosys / LibreLane)
Status	Simulation and post-layout simulation only. Nothing has been measured on silicon.

How to read this document. Every number is traceable to a file in the repository, a CI run, or a log named in the text. Where something is simulated, modelled, estimated or not done, the text says so in that place, and Section 9 gathers the limitations in one list. Section 8 maps each claim to the evidence behind it.

Contents

1. Executive summary	4
Headline figures	4
2. Goal and fit to the competition brief	5
Lineage	5
3. System architecture	6
3.1 CPU	6
3.2 Memory map (8-bit address space)	7
3.3 Boot and reprogramming	7
3.4 Pinout	7
4. The PIO block	8
4.1 Host interface and registers	8
4.2 Design decisions that shape the block	8
4.3 Host tooling	9
5. Protocol coverage	9
6. Physical implementation and portability	10
6.1 Hardening runs	10
6.2 Timing at 24 MHz	10
6.3 Portability: PDK-agnostic RTL, no macros	11
7. Verification	11
7.1 Method and what it can and cannot show	11
7.2 Test inventory	12
7.3 Mutation sweeps	13
7.4 Post-layout (gate-level) tests	13
7.5 Continuous integration	14
8. Claims and the evidence behind them	15
9. Limitations and open work	15
9.1 Architecture	15
9.2 Protocol-specific	16
9.3 Verification and evidence	16
9.4 Open work, in suggested order	16
10. Reproducing the results	17
Repository layout	17
Appendix A. Feature history	18

Sources used for this document 18

1. Executive summary

AgilA32 is a small system-on-chip built to answer one question: *can a chip learn new protocols after fabrication, with no host microcontroller?* It puts a from-scratch RV32I CPU next to two state machines that execute the RP2040 PIO instruction set. The CPU loads PIO programs from flash or from a GPIO bootloader, can replace them at run time, and can halt itself (EBREAK) while the PIO keeps the protocol running alone. On the RP2040 the programmer is an external Cortex-M0+; here it is on the die.

The design is plain synthesizable Verilog (about 2,900 lines in `src/`), uses no PDK cells and no macros, and fits the contest's 6x4-tile budget on IHP SG13 CMOS5L.

Headline figures

Item	Value	Where it comes from
Protocols demonstrated in simulation	UART TX/RX, SPI master modes 0-3, I2C master / slave / multi-master, JTAG, SWD, low-speed USB host, PS/2 receiver, WS2812 (two programs), 1-Wire, VGA 640x480	Section 5, README.md
Contest baseline (UART, SPI, I2C)	All three done, each with a CPU-driven end-to-end demo	Section 5
Stretch goals	Low-speed USB: done in simulation (host side, narrow receive tolerance). 10BASE-T: not attempted	Section 5, 9
PIO resources	2 state machines, 32-word shared instruction memory, 4-deep FIFOs, pins 0-9	<code>src/pio.v</code>
PIO programs in the repository	23 (19 protocol programs + 2 smoke-test + 2 signature-test programs)	<code>pio/</code>
Cocotb protocol tests	94 passing (76 + 18 SWD), GitHub runs of 4 min 28 s and 6 min 25 s	CI runs #10 (03013ea) and #16 (79f0997)
Standalone Icarus benches	31 sources, 33 runs (two WS2812 firmware variants re-run the same bench), all PASS	CI logs, 5 Oct 2026
Mutation testing	169 one-line mutants over seven sweeps: 167 caught; 2 survivors in the signature sweep (Section 7.3)	<code>tools/</code> (7 sweep scripts)
Gate-level tests	13/13 passing on the placed IHP netlist (5-6 Oct 2026, gds run #65), with the PIO smoke and signature tests; <code>gl_test</code> took 3 h 11 min (the earlier 12/12 run: 2,587 s)	Tiny Tapeout <code>gl_test</code> job, <code>results.xml</code>
Timing at 24 MHz	Worst setup slack +20.17 ns (slow corner); register-to-register limit about 46.5 MHz; hold +0.116 ns (fast corner)	GDS run, <code>docs/info.md</code>
Physical checks	0 Magic DRC, 0 routing DRC, 0 LVS errors, 0 antenna-violating nets	GDS run
Silicon measurements	None	-

What a reviewer should weigh. Strength: breadth of protocols on one small block, and a verification approach built around peer models, CPU-driven end-to-end benches and mutation testing. Weakness: RV32I plus an RP2040-style PIO is close to the RP2040's own architecture, every result is simulation, and only two small PIO tests have run on the post-layout netlist: a smoke test and a wider signature test (I2C pads, most instruction forms, autopush / autopull, side-set), which passed there on 5-6 Oct 2026. The protocol programs themselves are simulated on RTL only, and the gate-level job now takes about 3 hours (Section 7.4).

2. Goal and fit to the competition brief

The Jane Street brief asks for an open-source, reprogrammable protocol-emulator chip, inspired by the RP2040 PIO and the TI PRU, on IHP CMOS5L through Tiny Tapeout. UART, SPI and I2C are the baseline; low-speed USB and 10 Mbit Ethernet are stretch goals. Entries are judged on unique functionality and on novel design and verification methods. Area is limited to 6x4 tiles (8x4 may follow); the brief recommends SRAM over flip-flops for instruction storage and FPGA prototyping before submission.

Brief item	How AgilA32 responds	Status
UART, SPI, I2C	Pico SDK programs run unmodified; SPI modes 0-3 and I2C master/slave/multi-master added	Done (sim)
Low-speed USB	usb_ls.pio, 29 of 32 words, 1.5 Mb/s exactly at 24 MHz / CLKDIV 2; token, DATA and reply capture	Done (sim), host side only
10 Mbit Ethernet	Needs a frame buffer (4-deep FIFO holds 128 of about 576 bits), at least 40 MHz and external magnetics	Not attempted
Reprogrammable after fabrication	CPU rewrites instruction memory and registers at run time; one image runs UART, then SPI, then I2C on the same state machine	Done (sim)
Open source, IHP CMOS5L, 6x4 tiles	Verilog template, LibreLane flow, hardened at 6x4	Done
Verification novelty	Peer models, mutation sweeps, CPU-driven benches, post-layout smoke and signature tests	Done; no formal methods or constrained-random tests yet
SRAM for instruction storage	Not used: the PIO instruction memory is 32 x 16-bit flip-flops, so the design carries no macros	Deliberate trade-off (area vs portability)
FPGA prototype	An FPGA workflow exists in the repository; no FPGA demo against a real peer device has been run	Open

Lineage

AgilA32 is the 32-bit successor of AgilA8, the author's earlier 8-bit Tiny Tapeout project (github.com/alexandercoabad/AgilA8_microcontroller on Sky130, AgilA8_IHP for the IHP port). The QSPI engine, GPIO bootloader, bank-switched flash execution, Timer/PWM and generic SPI peripheral are ported from AgilA8. The RV32I core and the whole PIO block are new. The line started from Pineapple ONE, a RISC-V computer built from discrete 7400-series logic.

3. System architecture

The chip has three parts on one 8-bit memory-mapped bus: the CPU, the memory and peripheral decoder (mem.v) with its shared QSPI engine, and the PIO block. The address bus is only 8 bits wide because the design grew from Agila8; the PIO therefore sits behind a two-byte window.

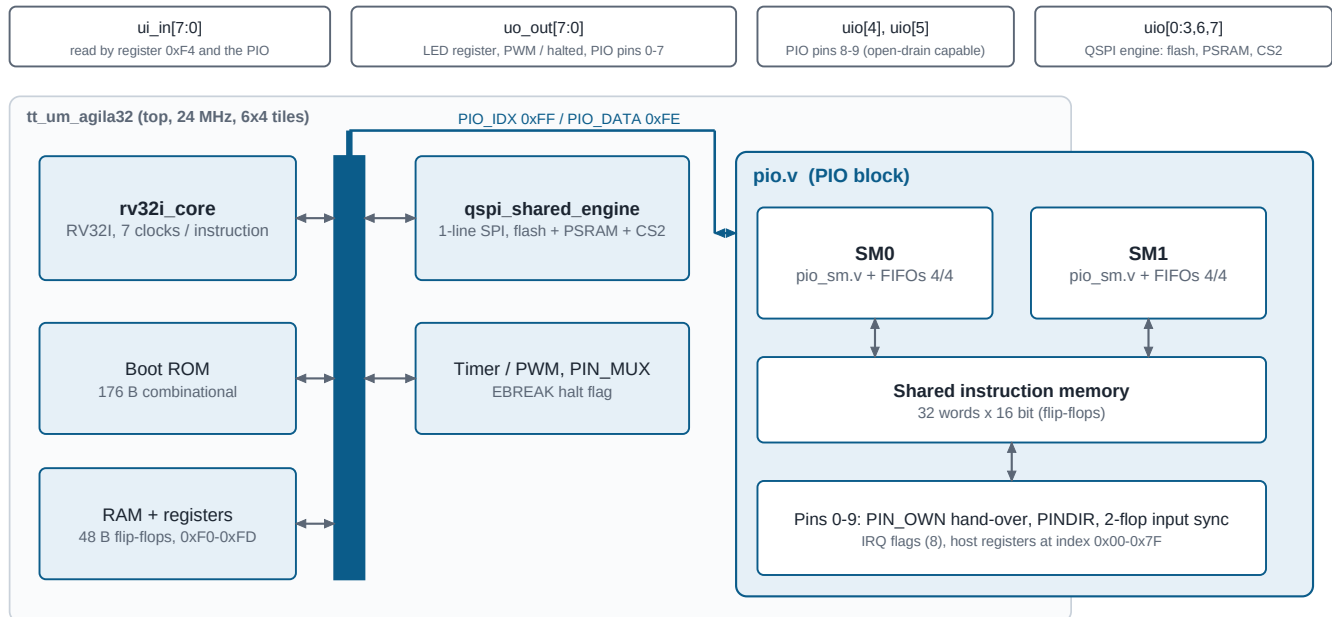


Figure 1. Block diagram. Pads are shared: pins are taken from the LED / QSPI logic only when the matching PIN_OWN bit is set.

3.1 CPU

- **Instruction set:** the full RV32I base integer set. FENCE, ECALL and EBREAK decode; there is no trap or exception handling. EBREAK is repurposed to park the core in a halted state until reset, and the halted flag can be routed to uo_out[7].
- **Micro-architecture:** multi-cycle FSM (fetch, fetch_wait, decode, execute, memory, mem_wait, writeback), 7 clocks per instruction. The two wait states let the core stall on a ready handshake during slow external QSPI accesses.
- **Cost of a flash fetch:** external flash is read through a 44-byte execute window with bank switching; one page switch costs about 3000 clocks, so the CPU feeds about one FIFO word per 3000 clocks. This sets the limit on CPU-fed protocol speed (Section 9).
- **Toolchain:** tools/asm_pineapple.py is a small RV32I assembler plus PagedAsm for paged flash images. tb_alu_test.v runs every opcode it wraps through the real core.

3.2 Memory map (8-bit address space)

Address	Item	Notes
0x00-0xAF	Boot ROM, 176 B	Combinational logic (a case statement), not flip-flops, so it is present at every power-on
0xB0-0xB3	Scratch RAM, 4 B	Always on-chip
0xB4-0xDF	Loadable / executable window, 44 B	Bootloader writes here; reads come from external flash (CS0) once FLASH_MODE is set
0xE0-0xEF	External PSRAM window, 16 B	RAM A (CS1); RAM B (CS2) when PSRAM_BANK = 1; core stalls until the SPI transaction completes
0xF0 / 0xF4	LED output / switch input	Wired to uo_out / ui_in; ui_in[0:2] are also the bootloader DATA / CLOCK / START
0xF1	PSRAM_BANK	Selects RAM A or RAM B for the 0xE0 window
0xF2, 0xF3, 0xF5, 0xF6	TIMER_LO, TIMER_HI, TIMER_CTRL, TIMER_FLAG	Free-running 16-bit timer (ported from AgilA8)
0xF7, 0xF9, 0xFA	PWM_DUTY, PWM_CTRL, PIN_MUX	8-bit PWM; PIN_MUX picks LED / PWM / halted on uo_out[7]
0xF8, 0xFC	FLASH_MODE, FLASH_PAGE	Redirect the window to flash; select the 44-byte flash page
0xFB	QSPI_CTRL	SCK = clk/2, /8, /32 or /128; resets to the slowest
0xFD	SPI_DATA	Raw 8-bit SPI on CS2; needs one board modification (cut a trace on the QSPI Pmod)
0xFE / 0xFF	PIO_DATA / PIO_IDX	PIO window: IDX bits [6:0] = register, bit 7 = auto-increment; DATA = 32-bit access

3.3 Boot and reprogramming

- **Self-test:** the boot ROM writes 0xA5 to the PSRAM window, reads it back, and latches the result into uo_out[7].
- **Listen loop:** a demo counter runs on uo_out[3:0] while START on ui_in[2] is polled. A bootloader request streams a program into RAM and runs it immediately, with no reflash.
- **Timeout fallback:** if START never arrives, the boot ROM sets FLASH_MODE itself and runs whatever is in external flash.
- **Flash hand-off:** a 4-byte stub bootloaded over the GPIO pins hands execution to flash; programs then page through flash with FLASH_PAGE.

3.4 Pinout

Pad	Function	Pad	Function
ui[7:0]	Switches; PIO input pins 0-7 (bootloader on ui[0:2])	uio[0..3]	QSPI CS0, MOSI, MISO, SCK
uo[6:0]	LEDs; PIO pins 0-6	uio[4], uio[5]	PIO pins 8, 9 (true bidirectional, held high until owned)
uo[7]	LED / PWM / halted (PIN_MUX); PIO pin 7	uio[6], uio[7]	QSPI CS1 (PSRAM A), CS2 (PSRAM B / generic SPI)

4. The PIO block

The block executes the RP2040 PIO instruction set: JMP, WAIT, IN, OUT, PUSH/PULL, MOV, IRQ and SET, with side-set, per-instruction delay, autopush and autopull, IRQ flags and a 16.8 fractional clock divider. Each state machine runs one instruction per divided-clock tick with deterministic timing, so a protocol is a short program loaded after fabrication.

Parameter	AgilA32	RP2040 (for comparison)
State machines	2 (parameter N_SM, 1 to 4)	8 (2 blocks of 4)
Instruction memory	32 words x 16 bit, shared by all state machines	32 words per block
FIFOs	4-deep TX and RX per state machine	4-deep, joinable to 8
Pins	0-9: uo_out[7:0], uio[4], uio[5]	30 GPIO
Input synchroniser	2 flip-flops, per-pin bypass (SYNC_BYP)	Same
Programmer	On-die RV32I over a two-byte window	External Cortex-M0+
Pad direction	PINDIR is the real output enable; PINDIR = 1 pulls the pad low (opposite to RP2040)	PINDIR = 1 drives the pad

4.1 Host interface and registers

The register map is in the header of `src/pio.v`. The index goes into `PIO_IDX`; with bit 7 set the index auto-increments after each `PIO_DATA` access, so a whole program streams in with back-to-back stores.

Index	Register	Content
0x00	CTRL	[3:0] SM_ENABLE, [7:4] SM_RESTART (write 1, self-clearing)
0x01 / 0x02	IRQ / IRQ_FORCE	8 flags; write 1 to clear / to set
0x03	FSTAT	RXFULL, RXEMPTY, TXFULL, TXEMPTY per state machine
0x04	PIN_OWN	[9:0] 1 = pin driven by PIO instead of the CPU / LED logic
0x05 / 0x06 / 0x07	SYNC_BYP / PINS_IN / PINS_OUT	Synchroniser bypass; input pins as seen by the SMs; output values and enables
0x08	INFO	N_SM, IMEM_DEPTH, FIFO_DEPTH, version
0x20-0x3F	IMEM	Instruction words 0-31
0x40 + 0x10 n	Per state machine n	CLKDIV, EXECCTRL, SHIFTCTRL, PINCTRL, INSTR, ADDR, TXF, RXF, FLEVEL

4.2 Design decisions that shape the block

- **PIN_OWN hand-over.** A pin leaves the LED / QSPI logic only when its `PIN_OWN` bit is set, so reset behaviour is identical to the pre-PIO chip and every earlier test passed unchanged. Hand-over order matters: `SCK` is preset in `GPIO_OUT` and a forced instruction must carry the idle level in side-set bit 12, or `SCK` glitches.
- **No pad-inversion override.** The Pico SDK gets `SPI CPOL = 1` by inverting the `SCK` pad; this block has no such override, so modes 2 and 3 are separate programs with inverted side-set values.
- **Open-drain buses on a real bidirectional pad.** Pins 8-9 use `PINDIR` as the output enable. Because the polarity differs from the RP2040, `pio/i2c.pio` has swapped side-set values and `tools/pio_i2c.py` complements the `pindir` bits it sends.
- **Designed for the synthesis flow.** The first CI synthesis stalled in Yosys' SAT-based share pass on variable-amount shifts in `pio_sm.v`. These were replaced by fixed-stage mux shifters with unchanged behaviour: the pass dropped from 145 analyses to 5 (28 s to 6 s locally) for about +2% generic gates.

- **Bounded programs by design.** A PIO state machine cannot take an asynchronous branch while it waits on a data bit, so the I2C slave receiver is fixed to a byte count at assembly time rather than risk going out of step on an early STOP.
- **No PDK cells, no macros.** Memories are Verilog arrays that synthesis maps to flip-flops. See Section 6.3.

4.3 Host tooling

`tools/pioasm.py` assembles stock pioasm syntax. `tools/pio_host.py` turns 'load this program, configure the state machine, start it' into a paged flash image, because each PIO register write costs several RV32I instructions and the on-chip execute window holds only 11 instructions. One `tools/build_pio_*.py` builder per demo produces the matching flash image.

5. Protocol coverage

Everything below runs as a PIO program (or as CPU firmware where noted). **All results are simulation.** 'Evidence' names the tests that back each row; Section 7 explains the methods. Word counts are of the 32 shared instruction words.

Protocol	Program (words)	Status	Evidence
UART TX	<code>uart_tx</code>	Done	Cycle-exact 8N1 waveform, integer / fractional / averaged dividers; CPU demo <code>tb_pio_cpu_uart.v</code>
UART RX	<code>uart_rx</code>	Done	TX+RX loopback, framing-error IRQ, baud tolerance, two-SM loopback
SPI master, modes 0-3	<code>spi_master</code> , <code>spi_cpha1</code> , <code>spi_cpol1_cpha0</code> , <code>spi_cpol1_cpha1</code>	Done	Data both ways, SCK idle level and edge count, SCK period, MOSI setup/hold, fast SCK with SYNC_BYP; CPU runs all four modes from one image with data relayed (<code>tb_pio_cpu_spi4.v</code>)
I2C master	<code>i2c</code>	Done	Write, read, repeated-start register read, NAK to IRQ 0, clock stretching; CPU demo <code>tb_pio_cpu_i2c.v</code>
I2C slave, both directions	<code>i2c_slave_rx</code> (29), <code>i2c_slave_tx</code> (32)	Done; write length fixed at assembly time; SCL period ≥ 32 clocks (use ≥ 40)	9 tests plus SCL-speed sweep; CPU swaps the two programs between a write and a read of the same device (<code>tb_pio_cpu_i2c_slave.v</code>); 29 mutants all caught
I2C multi-master	<code>i2c_mm</code> (27)	Done; no bus-busy detection	10 arbitration / clock-sync tests; 4 more document the bus-busy hazard (2 of 8 timings destroy the other master's transaction)
JTAG master	<code>jtag</code> (24)	Done	CPU resets the TAP, reads IDCODE, writes and reads back the USER register: 95 TCK pulses, pins-only TAP model checks every state
SWD host	<code>swd</code> (15)	Done at <code>pio.v</code> level; no CPU demo	18 tests against a protocol-strict SW-DP + MEM-AP model; 20 mutants all caught
Low-speed USB host	<code>usb_ls</code> (29)	Done; receive tolerance -0.5 % to +0.8 %	7 tests against a low-speed device model; CPU queues an IN token and halts, PIO does the transaction (<code>tb_pio_cpu_usb.v</code>)
PS/2 receiver	<code>ps2_rx</code> (8)	Done, CPU-driven end to end	7 tests incl. 4-frame burst, FIFO overflow, idle-gap resync; CPU sleeps while a 10 kHz keyboard model types
WS2812 / SK6812	<code>ws2812</code> (4)	Done; about 5 pixels per CPU-fed frame	5 tests with exact pulse widths (375 / 875 ns), RGBW; CPU demo with RGBW and 280 us-reset variants

Protocol	Program (words)	Status	Evidence
WS2812 long strip	ws2812_repeat (15)	Done; border LOW stretched to 1.0-1.5 us	300 pixels from one pair of FIFO words; CPU demo paints 60 pixels alone; 18 mutants all caught
1-Wire master	onewire (31)	Done; standard speed only	7 tests against a device model checking every pulse (reset, presence, slots, READ ROM + CRC-8); CPU demo; 23 mutants all caught
VGA 640x480	vga_frame (23) + vga_line (9)	Done; 8 colours; 57.1 Hz at 24 MHz	Monitor model counts HSYNC, VSYNC and 525 lines over 3 frames; CPU starts two SMS with one write and halts; 23 mutants all caught
PS/2 CPU bit-bang, ST7789 LCD	CPU firmware	Done (PS/2 only against a slowed keyboard)	tb_ps2_reader.v, tb_ps2_ascii.v, tb_st7789_driver.v
10BASE-T, CAN	-	Not attempted	Ethernet needs a frame buffer and at least 40 MHz

Reprogrammability demo. `tools/build_pio_multi.py` builds one flash image whose firmware reprograms the *same* state machine at run time: UART TX, then SPI mode 0, then I2C. Between phases it disables and restarts the machine, overwrites instruction memory and reconfigures pins, shifting and clock divider. `tb_pio_cpu_multi.v` checks all three waveforms against a UART receiver, an SPI slave and an I2C slave, and that the phases ran strictly in sequence. The firmware then executes EBREAK while the PIO finishes the I2C STOP alone.

6. Physical implementation and portability

6.1 Hardening runs

Run	Design	Result
Sky130, Tiny Tapeout CI (previous target)	CPU-only, 6x2 tiles	53.8 % routing utilisation, 12,548 cells (excl. fill/tap), precheck 15/15, gate-level 11/11
IHP, first run, 1 MHz target	CPU-only, 6x2 tiles	17,795 cells, 0.245 mm ² , about 62 % utilisation, 0 DRC / LVS / antenna violations, critical path about 206 ns, 5 max-slew violations (slow corner)
IHP SG13 CMOS5L, current target	CPU + PIO, 6x4 tiles	51.45 % routing utilisation, 34,053 cells (excl. fill/tap, as stated in the README), clean lint, precheck 10/10; the gds job took about 2 h 28 min (CI run #11)
IHP, 24 MHz period (41.667 ns)	CPU + PIO, 6x4 tiles	Cells 36,601 incl. 3,605 hold buffers (as stated in docs/info.md); identical synthesis, placement and routing to the 1 MHz run because nothing needed repair; extracted netlist has 3,433 flip-flops

The cell counts 34,053 and 36,601 come from two different documents and are quoted as each states them; I have not reconciled them against the raw run summary, and they probably differ in what they count. Treat both as 'about 35,000 cells'.

6.2 Timing at 24 MHz

Corner	Worst setup slack	Note
Slow (1.08 V, 125 C)	+20.17 ns	Worst path 22.1 ns arrival, register to register; limit about 46.5 MHz, roughly 1.9x margin at 24 MHz
Typical	+27.6 ns	TNS 0 in every corner
Fast	+29.6 ns	Worst hold slack +0.116 ns (this corner); no hold violations

Other checks in that run: 0 Magic DRC, 0 routing DRC, 0 LVS errors, 0 antenna-violating nets. Unchanged design-rule advisories (not failures): 44 max-slew violations (slow corner only), 258 max-fanout and 1 max-cap violation. The 24 MHz clock is chosen so the PIO tick is exactly 12 MHz at CLKDIV 2, which gives low-speed USB its

1.5 Mb/s with 8 ticks per bit.

6.3 Portability: PDK-agnostic RTL, no macros

- **No PDK dependency in the RTL.** `src/*.v` and `*.vh` are plain synthesizable Verilog. No standard cell, I/O cell or other library primitive is instantiated (every instantiated module is defined in `src/`, checked by scanning all module instantiations), and the only conditional compilation is ``ifndef SYNTHESIS` around simulation-only code.
- **No macros, no hard IP.** `src/config.json` lists no macros or extra files. All memories (16-bit PIO instruction memory, register file, 48 B RAM) are Verilog arrays that synthesis maps to flip-flops, not SRAM. The cost is area, which is why the instruction memory is 32 words. The gain is that any open flow and PDK can build the same RTL.
- **What carries over and what does not.** The CPU-only version was hardened on Sky130 and moved to IHP without an RTL change. The PIO block was added afterwards and has been hardened on IHP only, so its portability rests on generic RTL, not on a second PDK run. Timing and area figures belong to the IHP run; another PDK needs its own flow run.

7. Verification

7.1 Method and what it can and cannot show

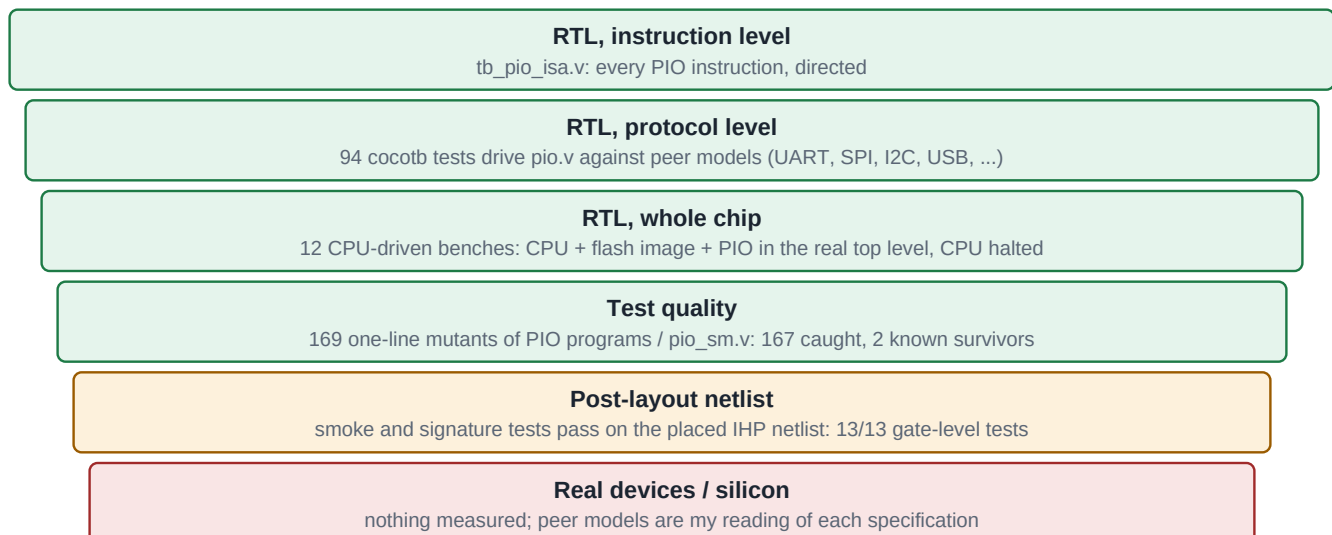


Figure 2. The evidence ladder. Green levels are repeatable in CI; amber is a single post-layout test; red is empty.

The central idea is to test against **peer models that check protocol events, not just waveforms**:

- The I2C slave model flags any SDA change while SCL is high, so START and STOP are checked as events.
- The SPI slave model measures MOSI setup and hold. A model that samples in the same cycle MOSI changes cannot tell a correct program from one whose data and clock edges coincide; swapping the mode programs makes the tests fail.
- The 1-Wire device model checks every pulse width against the specification windows; the SWD target ignores everything until the JTAG-to-SWD switch and checks request parity; the VGA monitor model counts every sync period over three frames.
- **CPU-driven benches** run the real CPU, a flash image and the PIO in the real top level, then check the waveforms from the pins alone, and that the CPU halted before the protocol finished.
- **Mutation sweeps** break a program one line at a time on a temporary copy and require some test to fail. This measures the tests, not the design.

Limits of the method. Every peer model is the author's reading of a specification, not a measured device. A model and a design that share a misreading would pass together. Mutation testing shows that the tests notice changes to the programs; it does not show the programs are right.

What verification found

- Three WAIT encodings in the supplied PIO testbench had the wrong source field (fixed, CHANGES_feature6.md).
- The first mutation runs of the 1-Wire and VGA sweeps each exposed a hole in the tests, now closed (CHANGES_feature15.md, CHANGES_feature16.md).
- The CPU's two-instruction delay loop takes about 274 clocks per pass, not the 14 the helper assumed (the first PS/2 demo slept 4.6 million clocks instead of 170,000).
- Bus-busy hazard in the I2C multi-master program measured: 2 of 8 START timings destroy the other master's transaction, and 'lost arbitration' is also reported in both destructive cases.
- The first real gate-level run failed 3 of 11 tests because a test helper forced a register by hierarchical name, which does not exist in the flattened netlist; the test was fixed, not the design (CHANGES_feature3_gl_fix.md). A separate Yosys error, 'multiple edge sensitive events' on one register in mem.v, was an RTL coding problem and was fixed in the RTL (CHANGES_feature4_gl_fix.md).

7.2 Test inventory

Suite	Count	Runs in	Latest result
On-chip cocotb (test/test.py + smoke and signature tests)	13 tests (12 + signature test)	test.yaml (as make), gds.yaml gate-level job	13/13 on RTL (local run) and 13/13 on the placed netlist (gds run #65, 3 h 11 min for the job)
PIO protocol cocotb (test_pio_protocols.py, test_pio_swd.py)	94 tests (76 + 18)	pio-tests.yaml	94/94, 0 failures, 0 errors, 0 skipped
Standalone Icarus benches	31 sources: 15 CPU / peripheral, 12 CPU-driven PIO, 4 pio.v-direct	pio-tests.yaml	33 runs, all end in PASS
Mutation sweeps	169 mutants in 7 sweeps	Manual workflow dispatch; the signature sweep was run locally	167 caught, 2 survivors (see 7.3)

7.3 Mutation sweeps

Sweep (tools/)	Target	Mutants	Examples of what is broken
i2c_slave_mutation_sweep.py	i2c_slave_rx.pio, i2c_slave_tx.pio	29	no address compare, ACKs a read request, one byte too many or too few, never drives ACK, any SDA fall counts as START, no clock stretching, ignores the master's NAK
onewire_mutation_sweep.py	onewire.pio	23	reset low 464 us (below 480), presence sampled at 37 us, write-0 low 33 us, read sampled 20 us after the fall, data bit inverted, bit count off
vga_mutation_sweep.py	vga_frame/line.pio	23	HSYNC 95 ticks, VSYNC one line, 479 picture lines, bar 79 ticks, no black after the last bar, palette not reloaded each line
ws2812_repeat_mutation_sweep.py	ws2812_repeat.pio	18	0-bit high 2 ticks, pixel never ends, run never ends, count word ignored, line left HIGH while idle
swd_mutation_sweep.py	swd.pio	20	read sampled one tick early, SWCLK high 3 ticks, SWDIO stays an output, read flag inverted, nothing pushed, data two bits per clock
gl_smoke_mutation_sweep.py	smoke-test programs and pio_sm.v	16	no invert, no bit-reverse, no push, wrong IRQ, 3 pulses, wrong widths, jmp x-- not decrementing
gl_sig_mutation_sweep.py	signature-test programs, pio_sm.v, pio.v	40 (38 caught)	shift direction, autopush threshold, out exec ignored, jmp conditions inverted, wrong IN_BASE, side-set not PINDIRS, side-set enable ignored, no synchroniser bypass, one trip short. Survivors: autopull threshold boundary, STATUS TX/RX select

7.4 Post-layout (gate-level) tests

`test/test_pio_gl.py::test_pio_postlayout_echo_and_pulses` runs in the Tiny Tapeout gate-level job. Through the real boot path (GPIO bootloader, hand-off stub, flash) it runs a 22-page image built by `tools/build_pio_gl_smoke.py`. SM0 (`gl_echo.pio`) pulls a word, inverts and bit-reverses it and pushes it back. SM1 (`gl_pulses.pio`) waits for IRQ 0 and sends bursts of 4 pulses on pad `uo_out[7]`. The CPU feeds 0x55, 0x01, 0xF0; the test checks the three echoes in order, exactly 12 pulses each 4 clocks high with a 10-clock period inside each burst, a low pad at the end, and the expected `uio_oe`.

Result (smoke test): the first gate-level run passed 12/12 tests in 2,587 s (the smoke test is about 10 s on RTL); in run #65 it took 3,444 s. That makes the job about 50 minutes instead of about 6 for the smoke test alone. Run time depends on the hosted machine, so treat the figure as one measurement.

Exercised by the smoke test	NOT exercised by the smoke test
pull block, mov x ~osr, mov isr ::x, push block, irq set 0, wait 1 irq 0, set x, set pins with delay, jmp x--; two state machines (CLKDIV 1 and 2); PIN_OWN on pin 7; CPU-to-PIO bus handshake; both FIFO directions	in; every out form; set pindirs; side-set; every jmp condition except x--; wait on pins; out exec / mov exec; mov from pins or status; autopush / autopull and thresholds; left-shift direction; irq wait / clear / relative; input synchronisers; pads 8 and 9 (uio_oe never changes)

Consequence. The I2C, USB and SWD programs rely on exactly the instruction forms and open-drain pads that the smoke test never touches. The signature test below covers those forms and pads and has passed on the placed netlist; the I2C, USB and SWD programs themselves have still not run there.

Signature test (feature 20). `test/test_pio_gl_sig.py::test_pio_postlayout_signature` is a second test in the same two jobs (`COCOTB_TEST_MODULES = test, test_pio_gl, test_pio_gl_sig`). The CPU loads `gl_sig0.pio` (17 words) and `gl_sig1.pio` (14 words) with `tools/build_pio_gl_sig.py` and owns all ten

pins; the test compares the pads with golden values worked out by hand and confirmed on RTL. SM0 shows each result on `uo_out [7:0]`: set, mov (plain, ~, ::), in pins / x (left shift), out pins / y (left shift), pull, push, irq set rel, jmp x--, and, through eight host-supplied out-exec words, mov from ~y / isr / status / y, set pins, in y, jmp x!=y, jmp !x (not taken), jmp pin, jmp y--, wait gpio and wait pin with `IN_BASE` (a taken jump skips a sentinel that would show on the pads). SM1 drives the open-drain pads 8 and 9 with side-set `PINDIRS`, reads them back through the input synchroniser (pad 8 bypassed, pad 9 synchronised) and uses autopull (threshold 16) and autopush (threshold 8).

Result (signature test): passed on the placed netlist in gds run #65 (5-6 Oct 2026): gate-level 13/13, 7,564 s for this test. The `gl_test` job took 3 h 11 min and the whole workflow 5 h 52 min, inside GitHub's 360-minute limit per job. The test checks the exact `uo_out` sequence, the exact (output enable, output value) sequence on the pads with testbench pull-ups, the RX words (0x93, 0xC3, 0x5A and the pushed ISR) and the IRQ flags.

Evidence for the signature test	Result
RTL (local run, 5 Oct 2026)	Passes; full make in test / 13/13; about 112,000 clocks
Yosys-synthesized netlist of <code>src/</code> (generic gates, hierarchy kept; not the hardened IHP netlist)	Passes
Cost on the placed netlist (gds run #65)	7,564 s and 4.40 million simulated clocks, against 3,444 s and 1.97 million for the smoke test in the same run: about 2.2 times (my RTL-based estimate of 1.9 was low)
Mutation sweep (40 mutants)	38 caught. Survivors: autopull threshold boundary; STATUS TX/RX select
Run on the placed IHP netlist	Passed (gds run #65, 5-6 Oct 2026, push 79f0997): gate-level 13/13, 0 failures; <code>gl_test</code> 3 h 11 min, whole workflow 5 h 52 min

Still not covered by either test: `jmp ly`, `out pindirs / pc / isr`, `mov exec / pc / isr / osr` destinations, `push iffull`, `pull ifempty`, a wait that really stalls on a pin, and the latency difference between bypassed and synchronised pins (the delays leave margin).

7.5 Continuous integration

Workflow	What it runs	Last result
<code>test.yaml</code>	make: on-chip cocotb tests including the PIO smoke and signature tests (RTL)	Result not reviewed for this document
<code>gds.yaml</code>	Synthesis, place and route, precheck, gate-level tests	Run #65 (push 79f0997, 5-6 Oct 2026): success, 5 h 52 min; gds 2 h 40 min, <code>gl_test</code> 3 h 11 min, precheck 8 min (the total is about gds plus <code>gl_test</code> , so precheck overlaps); gate-level 13/13. Earlier run: 12/12, 2,587 s of test time
<code>pio-tests.yaml</code>	make standalone-tests: 31 benches + 94 protocol tests; mutation sweeps only on manual dispatch	Run #16 (push 79f0997): success, 6 min 25 s, 94/94, 33 bench logs PASS. Run #10 (03013ea): 4 min 28 s, 94/94. Run #2 (5de56c9): 76/76, 9 min 4 s
<code>fpga.yaml</code> , <code>docs.yaml</code>	FPGA build, documentation build	Not reviewed for this document

Three runs of the same workflow took 9 min 4 s, 4 min 28 s and 6 min 25 s, so quoted run times depend on the hosted machine. Before `pio-tests.yaml` existed on GitHub, the PIO benches and the 76-test protocol suite had run only in local simulation, outside GitHub; the first GitHub run (4 Oct 2026) is the first independent confirmation.

8. Claims and the evidence behind them

Strength key: **CI** = reproduced in a GitHub Actions run; **Sim** = simulation run by the author, repeatable with the commands in Section 10; **Model** = checked against the author's own peer model only; **Estimate** = derived or extrapolated; **None** = not done.

Claim	Evidence	Strength
RV32I core runs all base instructions	<code>tb_alu_test.v</code> (every opcode the assembler wraps), on-chip cocotb tests	CI (on-chip), Sim
Chip reprograms its protocol engines at run time, no host MCU	<code>tb_pio_cpu_multi.v</code> , <code>tb_pio_cpu_spi4.v</code> , <code>tb_pio_cpu_i2c_slave.v</code>	CI, Model
Protocol keeps running with the CPU halted	CPU-driven benches check the CPU halted before the last pulse / byte (UART, I2C, USB, JTAG, VGA, WS2812)	CI, Model
UART, SPI, I2C baseline met	Section 5 rows, 94 cocotb tests	CI, Model
Low-speed USB host transaction works	<code>usb_ls.pio</code> against <code>UsbLsDevice</code> ; no real device	CI, Model
Timing holds at 24 MHz	GDS run: setup +20.17 ns slow corner, hold +0.116 ns fast corner	GDS run (CI)
Design is DRC / LVS clean	0 Magic DRC, 0 routing DRC, 0 LVS errors, 0 antenna violations	GDS run (CI)
Post-layout netlist behaves like RTL	13/13 gate-level tests, including the PIO smoke and signature tests (no protocol program has run there)	CI, narrow
Tests would catch regressions in PIO programs	169 mutants, 167 caught (2 listed survivors)	Sim (manual workflow, local)
RTL is PDK-agnostic and uses no macros	Instantiation scan, config review, Sky130 to IHP move without RTL change (CPU-only)	Sim, review
Fits 6x4 tiles	51.45 % utilisation on IHP at 6x4	GDS run (CI)
Works against real peer devices	-	None
Works on silicon	-	None
Formal verification / constrained-random	-	None

9. Limitations and open work

These are collected from the project's own documentation.

9.1 Architecture

- Two state machines (RP2040: eight), 32 shared instruction words, pins 0-9 only. Programs that need more words cannot run together: the two I2C slave programs use 29 + 32 words, so a combined register-file slave (write a pointer, repeated START, read) is not possible; the CPU swaps programs instead.
- The CPU has no interrupts and no traps, executes an instruction in 7 clocks, and feeds about one FIFO word per 3000 clocks (flash paging). Protocols must be slower than that per byte, or fit in the FIFOs.
- The address bus is 8 bits wide; the external window is 16 bytes. Reaching the QSPI Pmod's full capacity needs a wider bus throughout the core.
- The generic SPI peripheral on CS2 needs one board modification (a trace cut on the QSPI Pmod). Until then it is inert.

9.2 Protocol-specific

Protocol	Limit
I2C slave	Write direction is bounded to a byte count fixed at assembly time. SCL period must be at least 32 clocks (40 or more advised). No combined slave.
I2C multi-master	No bus-busy detection before START: 2 of 8 timings destroy another master's transaction, and arbitration loss is reported even then. The host must wait for a free bus, but the check and START are separate steps (3000-clock page switch in between). No stuck-bus recovery. The program has 5 spare words; a PIO-side check was not tried.
Low-speed USB	Receive tolerance only -0.5 % to +0.8 % (spec allows +-1.5 %), so RC-oscillator devices may fail on long packets. TX is FIFO-sized (96 bits), so a SETUP + 8-byte DATA0 transfer is not possible from CPU firmware. Host side only; electrical levels not measured.
PS/2	Host-to-device commands (LED set, reset, inhibit) not done; they need open-drain CLOCK/DATA on pins 8-9. The keyboard model is the author's reading of the timing.
WS2812	CPU-fed frames limited to about 5 pixels; the repeat program stretches border LOW times to 1.0-1.5 us, outside the nominal windows. Not tried on a strip.
1-Wire	Standard speed only; no overdrive or strong pull-up; usable clock window about -7.7 % to +15 %.
VGA	8 colours at about 2/3 brightness; 57.1 Hz at 24 MHz (59.94 Hz needs 25.175 MHz); no frame buffer; no real monitor.
JTAG / SWD	SWD: no CPU-driven demo, no multi-drop, no dormant sequence, not tried on a target (needs an external pull-up on SWDIO). JTAG: model TAP only.
10BASE-T, CAN	Not attempted.

9.3 Verification and evidence

- All results are simulation. Nothing is measured on silicon, on a real peer device, or on a real board.
- Peer models are the author's interpretation of each specification.
- No formal verification and no constrained-random testing yet.
- Only two small PIO tests have run on the placed netlist (smoke and signature); the protocol programs themselves have not (Section 7.4). The gate-level job now takes about 3 hours.
- The external-memory path is tested only against a hand-written flash / PSRAM model.
- Quoted CI and gate-level run times vary with the hosted machine.

9.4 Open work, in suggested order

- Formal properties for the PIO (FIFO and shifter invariants first) and a constrained-random protocol test.
- Cut the gate-level run time (the signature test costs about 2 hours), then add a post-layout I2C transaction test.
- An FPGA demonstration against one real peer device (a keyboard, a USB device or an I2C sensor).
- A PIO-side bus-free check for the I2C multi-master program; per-transition resync for USB receive.
- A CPU-driven SWD demo, and a decision on 10BASE-T (the most visible stretch goal, but only if it can be finished and verified).
- Step 3 of the original plan: a PIO glyph expander plus renderer tying the PS/2 keyboard to the ST7789 display.

10. Reproducing the results

Requirements: Icarus Verilog (`apt-get install iverilog`), Python 3.11 and `pip install -r test/requirements.txt` (cocotb 2.1.0 was used).

```
cd test
make                # on-chip cocotb: self-test, demo counter, bootload-and-run, PIO smoke and signature tests
make standalone-tests # 31 Icarus benches, then the 94 cocotb protocol tests (Makefile.proto)
make -f Makefile.proto COCOTB_TEST_FILTER=test_owewire_read_rom # one protocol test

# mutation sweeps (each runs the tests once per mutant; add --list to see the mutants)
python3 tools/i2c_slave_mutation_sweep.py # 29 mutants
python3 tools/owewire_mutation_sweep.py   # 23
python3 tools/vga_mutation_sweep.py       # 23
python3 tools/ws2812_repeat_mutation_sweep.py # 18
python3 tools/swd_mutation_sweep.py       # 20
python3 tools/gl_smoke_mutation_sweep.py  # 16
python3 tools/gl_sig_mutation_sweep.py    # 40 (38 caught)
```

The longest single protocol test (`test_owewire_clock_tolerance`) takes about 80 s. In CI, `pio-tests.yaml` runs on every push and fails on any FAIL log or `<failure` in `results.xml`. The gate-level run needs the Tiny Tapeout hardening flow and happens in the `gds.yaml` workflow.

Repository layout

Path	Content
<code>src/</code>	RTL: <code>rv32i_core.v</code> , <code>mem.v</code> , <code>qspi_shared_engine.v</code> , <code>pio.v</code> , <code>pio_sm.v</code> , <code>pio_fifo.v</code> , <code>tt_um_agila32.v</code> (about 2,900 lines with headers), generated <code>boot_rom_body.vh</code> , <code>config.json</code>
<code>pio/</code>	21 PIO programs
<code>tools/</code>	Assemblers (<code>asm_pineapple.py</code> , <code>pioasm.py</code>), host library (<code>pio_host.py</code>), flash-image builders (<code>build_*.py</code>), protocol helpers, mutation sweeps, <code>sta.py</code> (quick pre-layout timing estimate)
<code>test/</code>	Cocotb tests, 31 standalone benches, peer models (<code>pio_tb_lib.py</code>), flash images, Makefiles
<code>docs/info.md</code>	Project datasheet (how it works, how to test, pinout, per-protocol detail)
<code>CHANGES_feature*.md</code>	One record per feature, including limits and mutation results
<code>MIGRATION_TO_IHP.md</code>	Sky130 to IHP notes
<code>.github/workflows/</code>	CI workflows

Appendix A. Feature history

The project grew by numbered features; each has a `CHANGES_featureN.md` with what changed, the tests, the mutation checks and the stated limits.

#	Feature
1	Halted-status PIN_MUX mode (ported from AgilA8)
2	Boot timeout, automatic flash fallback (ported)
3	Variable SPI clock divider (ported); followed by RTL and gate-level fixes (3/11 to 11/11)
4	General-purpose SPI peripheral on CS2 (ported); followed by a gate-level fix for a multiple-edge-event error
5	Dual-port RAM area merge: investigated, not applicable
6	PIO block: firmware-defined protocol emulation (UART, SPI mode 0, I2C), 26 cocotb tests at the start
7	SPI master modes 2 and 3 (CPOL = 1)
8	CPU-driven demo of all four SPI modes
9	CPU-driven JTAG (IEEE 1149.1)
10	PS/2 receiver on the PIO, and an honest redesign of the keyboard-to-display step
11	Low-speed USB host engine; core clock raised to 24 MHz
12	I2C slave (both directions) and arbitrating multi-master
13	CPU-driven top-level demo of the PS/2 receiver
14	WS2812 / SK6812 LED strips
15	1-Wire master
16	VGA 640x480 colour bars on the Tiny VGA Pmod
17	WS2812 extras: repeat-colour program, RGBW and 280 us reset variants; PIO test workflow
18	Post-layout (gate-level) PIO smoke test
19	SWD (ARM Serial Wire Debug) host on the PIO
20	Post-layout (gate-level) PIO signature test

Sources used for this document

- `README.md`, `docs/info.md`, `info.yaml`, `src/config.json`, `MIGRATION_TO_IHP.md` and `CHANGES_feature*.md` of the 5 Oct 2026 repository snapshot.
- GitHub Actions results: `pio-tests` run #16 (push 79f0997), run #10 (push 03013ea) and the earlier run #2 (push 5de56c9), the `results.xml` and bench logs of those runs, and the Tiny Tapeout gate-level `results.xml` (first run: 12 test cases; gds run #65 of 5-6 Oct 2026: 13 test cases with per-test times, 0 failures).
- Competition brief: Jane Street, 'Protocol emulator ASIC competition' (blog.janestreet.com/protocol-emulator-asic-competition/).